Python

Pada kesempatan ini, kita akan mempelajari beberapa hal terkait pemograman lanjutan python seperti:

1. Object Oriented Programming (OOP) di Python
2. Modular Code
3. Efficiency Code
4. Best Practice dalam Python Coding

Object Oriented Programming (OOP) di Python

Object Oriented Programming atau yang biasa disingkat dengan OOP adalah suatu paradigma pemograman yang berfokus pada objek. Dalam pemograman OOP, kita memandang suatu program berisikan objek-objek yang saling mendukung dan berinteraksi satu sama lain. Tidak semua bahasa pemograman dapat mendukung paradigma pemograman ini dan Python adalah salah satu bahasa pemograman yang mendukung OOP.

Week 4 - Main Principles of OOP

Enkapsulasi

In [1]:
```
class User:
    def __init__(self, username, email, gender):
        self._username = username  # Enkapsulasi dengan menggunakan underscore (_)
        self._email = email
        self._gender = gender

    def get_username(self):
        return self._username

    def set_username(self, username):
        # Validasi atau logika tambahan dapat ditambahkan di sini
        self._username = username
```

In [2]:
```
# Contoh penggunaan
user = User("Jonathan Adis Pratama", "Jonathan@example.com", "Male")
print(user.get_username())  # Mengakses melalui metode getter
user.set_username("Jo saja")  # Mengubah melalui metode setter
print(user.get_username())
```

```
Jonathan Adis Pratama
Jo saja
```

1. Enkapsulasi digunakan untuk menyembunyikan implementasi internal dan melindungi atribut agar tidak dapat diakses atau diubah secara langsung.
2. Atribut username, email, dan gender diberi awalan _ untuk menandakan bahwa seharusnya diakses secara terbatas.
3. Metode getter get_username digunakan untuk mengakses nilai atribut, dan setter set_username digunakan untuk mengubahnya dengan validasi atau logika tambahan.

Hands On

dari program diatas kita bisa mengubah username menggunakan method get dan set, lalu bagaimana jika saya ingin mengubah gender "Male" dari user Jonathan menjadi "unknown"???

In [3]:
```
class User:
    def __init__(self, username, email, gender):
       self._username = username  # Enkapsulasi dengan menggunakan underscore (_)
       self._email = email
       self._gender = gender

    def get_username(self):
        return self._username

    def set_username(self, username):
        # Validasi atau logika tambahan dapat ditambahkan di sini
         self._username = username

    def get_gender(self):
        return self._gender

    def set_gender(self, gender):
        self._gender = gender
```

In [4]:
```
user = User("Hafiz Padlur Rizqan", "hafizpadlur@example.com", "Male")

print(user.get_username())
user.set_username("Hafiz")
print(user.get_username())

print(user.get_gender())      # menampilkan gender awal
user.set_gender("unknown")   # mengubah gender
print(user.get_gender())
```

```
Hafiz Padlur Rizqan
Hafiz
Male
unknown
```

Inheritance

Dalam OOP, dikenal suatu istilah yang bernama inheritance atau pewarisan. Istilah ini mengisyaratkan bahwa kita dapat membuat suatu kelas baru yang berasal dari turunan kelas yang lain.

Kelas yang merupakan turunan (atau children/anak) dari kelas lainnya (biasa disebut dengan parent) akan mewarisi attribut dan method yang berasal dari parentnya.

Child class juga dapat melakukan overriding terhadap method yang terdapat dari parent class.

In [5]:
```
class PremiumUser(User):
    def __init__(self, username, email, gender, premium_level):
        super().__init__(username, email, gender)
        self.premium_level = premium_level

    def display_premium_info(self):
        print("Premium Level:", self.premium_level)
```

In [6]:
```
premium_user = PremiumUser("Jonathan Adis Pratama", "Jonathan@example.com", "Male",
premium_user.display_premium_info()
```

```
Premium Level: Gold
```

1. Inheritance memungkinkan kelas anak (PremiumUser) untuk mewarisi atribut dan metode dari kelas induk (User).
2. Melalui super().**init**, konstruktor kelas anak memanggil konstruktor kelas induk untuk menginisialisasi atribut yang sama.
3. Kelas anak dapat menambahkan metode tambahan seperti display_premium_info

Polymorphism

In [7]:
```
class User:
    def __init__(self, username, email, gender):
        self.username = username
        self.email = email
        self.gender = gender

    def display_user_info(self):
        print("Username:", self.username)
        print("Email:", self.email)
        print("Gender:", self.gender)

class PremiumUser(User):
    def __init__(self, username, email, gender, premium_level):
        super().__init__(username, email, gender)
        self.premium_level = premium_level

    def display_user_info(self):  # Override metode display_user_info dari kelas Us
        super().display_user_info()  # Memanggil metode display_user_info dari kela
        print("Premium Level:", self.premium_level)
```

In [8]:
```
def display_user_information(user):
    if isinstance(user, User):
        user.display_user_info()
    else:
        print("Objek tidak dikenali.")
```

In [9]:
```
# Contoh penggunaan
user1 = User("Jonathan Adis Pratama", "Jonathan@example.com", "Male")
user2 = PremiumUser("Jonathan Adis Pratama", "Jonathan@example.com", "Male", "Gold"

display_user_information(user1)  # Memanggil metode display_user_info dari kelas Us
display_user_information(user2)  # Memanggil metode display_user_info dari kelas Pr
```

```
Username: Jonathan Adis Pratama
Email: Jonathan@example.com
Gender: Male
Username: Jonathan Adis Pratama
Email: Jonathan@example.com
Gender: Male
Premium Level: Gold
```

Dalam contoh code, display_user_information berperan seperti remote kontrol tersebut. Meskipun kita mungkin memiliki objek dari kelas User dan PremiumUser, fungsi ini mampu "mengendalikan" keduanya dengan cara yang sama, yaitu memanggil metode display_user_info. Hal ini mencerminkan prinsip polimorfisme di mana suatu fungsi dapat bekerja dengan objek-objek yang berbeda dengan menggunakan antarmuka yang seragam.

Dengan polimorfisme, kita dapat menulis kode yang lebih fleksibel dan mudah diubah, karena tidak perlu tahu tipe objek secara spesifik, tetapi cukup tahu bahwa objek tersebut memiliki antarmuka yang diperlukan.

Abstraction

In [10]:
```
import abc
print(dir(abc))
```

```
['ABC', 'ABCMeta', '__builtins__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', '_abc_init', '_abc_instancecheck', '_abc_register', '_abc
_subclasscheck', '_get_dump', '_reset_caches', '_reset_registry', 'abstractclassmeth
od', 'abstractmethod', 'abstractproperty', 'abstractstaticmethod', 'get_cache_toke
n', 'update_abstractmethods']
```

- *ABC* digunakan untuk membuat kelas abstrak, yaitu kelas yang tidak bisa langsung dibuat objeknya.
- *abstractmethod* digunakan untuk mendeklarasikan method abstrak, yaitu method yang harus diimplementasikan oleh subclass.

In [11]:
```
from abc import ABC, abstractmethod

# Kelas abstrak (abstraksi) untuk jenis mobil
class JenisMobil(ABC): #kelas yang diwarisi oleh ABC
    def __init__(self, nama, warna):
        self.nama = nama
        self.warna = warna

    @abstractmethod
    def informasi(self):
        pass

# Kelas turunan yang mengimplementasikan metode abstrak dari JenisMobil
class Sedan(JenisMobil):
    def informasi(self):
        return f"Sedan {self.nama} berwarna {self.warna}"

# Kelas turunan lainnya
class SUV(JenisMobil):
    def informasi(self):
        return f"SUV {self.nama} berwarna {self.warna}"

# Fungsi untuk menampilkan informasi mobil
def tampilkan_informasi(mobil):
    print(mobil.informasi())
```

In [12]:
```
# Contoh penggunaan
sedan_mobil = Sedan("Civic", "Merah")
suv_mobil = SUV("CR-V", "Hitam")

tampilkan_informasi(sedan_mobil)
tampilkan_informasi(suv_mobil)
```

```
Sedan Civic berwarna Merah
SUV CR-V berwarna Hitam
```

A. Abstraksi:

```
- Kelas abstrak JenisMobil menyediakan kerangka kerja untuk jenis
mobil apa pun. Atribut seperti nama dan warna adalah aspek-aspek
abstrak yang relevan untuk semua jenis mobil.
- Metode abstrak informasi memungkinkan setiap kelas turunan untuk
memberikan implementasi khususnya sesuai dengan jenis mobil yang
mereka wakili.
```

B. Penggunaan Abstraksi:

```
- Penggunaan fungsi tampilkan_informasi menunjukkan bagaimana kita
dapat menggunakan antarmuka umum (informasi) untuk menampilkan
informasi mobil tanpa perlu tahu detail implementasinya.
- Pengguna luar hanya perlu tahu bahwa objek dapat memberikan
informasi tentang jenis mobil, dan detail implementasi dikurangi.
```

Hands On

Buatlah program yang mengandung 2/3 Main Principles of OOP selain contoh diatas. Lalu presentasikan kedepan satu-satu

In [13]:

```python
class Robot:
    def __init__(self, name, energy):
        self._name = name          # Enkapsulasi
        self._energy = energy

    def move(self):
        print(self._name, "is moving")

    def display_info(self):
        print("Robot Name:", self._name)
        print("Energy:", self._energy)


class Drone(Robot):
    def __init__(self, name, energy, altitude):
        super().__init__(name, energy)
        self.altitude = altitude

    def move(self):
        print(self._name, "is flying at", self.altitude, "meters")


robot1 = Robot("NexusBot", 80)
robot2 = Drone("SkyDrone", 100, 50)

robot1.display_info()
robot1.move()

print()

robot2.display_info()
robot2.move()
```

```
Robot Name: NexusBot
Energy: 80
NexusBot is moving

Robot Name: SkyDrone
Energy: 100
SkyDrone is flying at 50 meters
```